

1.了解那些激活函数？详细讲一下你用到的激活函数GeLU

常见的激活函数包括 Sigmoid、ReLU、Leaky ReLU、Tanh 等。

- Sigmoid 函数将输入值映射到 0 到 1 之间，常用于二分类问题。
- ReLU 函数在输入大于 0 时保持原样，小于 0 时为 0，它在深度神经网络中广泛使用。
- Leaky ReLU 是 ReLU 的一种变体，对于小于 0 的输入，它会有一个小斜率，避免了神经元的死亡。
- Tanh 函数将输入值映射到-1 到 1 之间，类似于 Sigmoid，但具有更平滑的过渡。

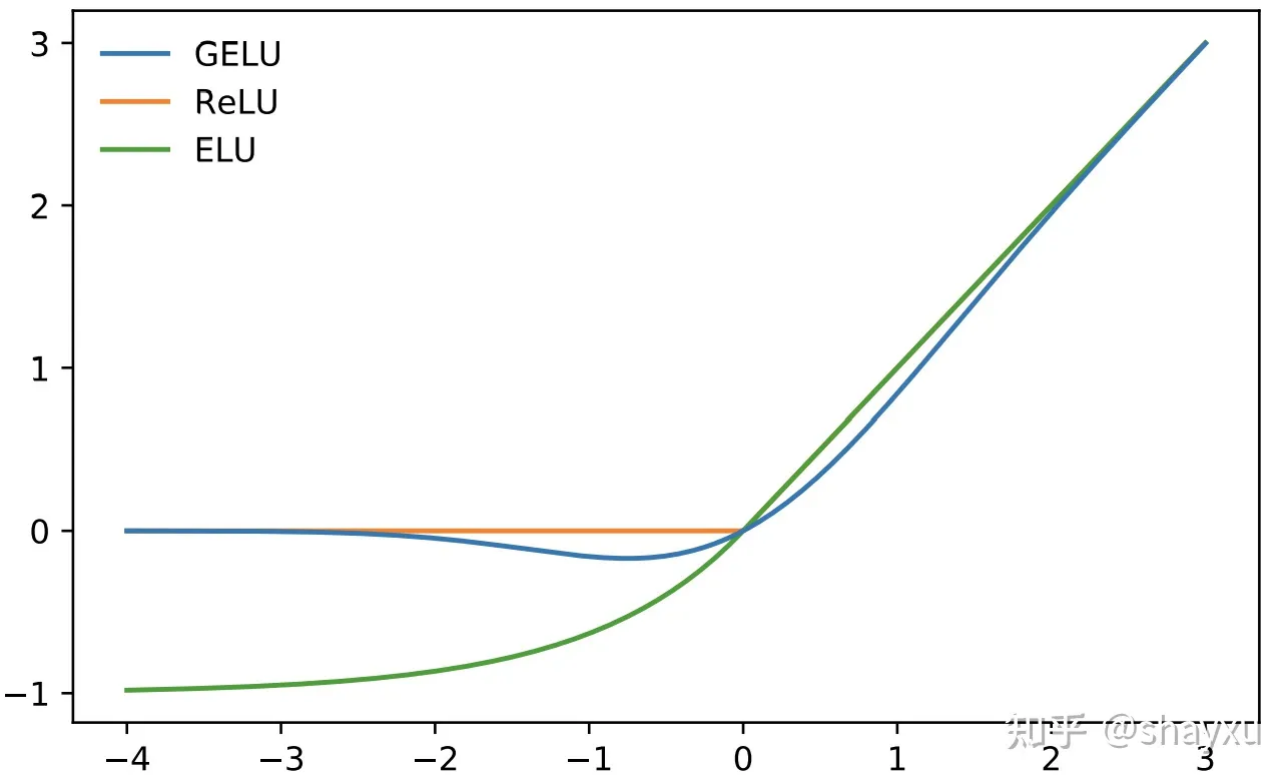
关于 GeLU（Gaussian Error Linear Unit）激活函数，它是一种近来在深度学习中受到关注的激活函数。GeLU 函数的定义如下：

$$GELU(x) = \frac{1}{2} [1 + \tanh(\sqrt{\frac{2}{\pi}} (x + 0.047715x^3))]$$

或者

$$GELU(x) = x * \sigma(1.702x)$$

其中， GeLU 函数在输入值较大时具有类似 ReLU 的效果，而在输入值接近零的时候具有更平滑的过渡。这使得它在一些情况下能够提供更好的性能，尤其是在处理 tjd/uuid 数据时。



GeLU的优点

它在处理负数时不会像ReLU一样将输入裁剪到0，这可能导致梯度消失的问题。

- 具有更光滑的导数：
 - GELU函数的导数是连续的，这使得在训练深度神经网络时可以更容易地传播梯度，避免了ReLU函数在 0 处的导数不连续的问题，从而减少了训练过程中出现的梯度消失问题
- 可以加速收敛：
 - GELU函数在激活函数的非线性变换中引入了类似于sigmoid函数的变换，这使得GELU函数的输出可以落在一个更广的范围内，有助于加速模型的收敛速度。

2.项目的构建过程？

项目的构建过程通常包括以下步骤：

- **问题定义**：明确项目的目标 and 需求，确定要解决的问题。
- **数据收集**：收集与问题相关的数据，可能需要从多个来源获取数据，并进行数据清洗和预处理。
- **数据预处理**：对数据进行清洗、转换、缩放等操作，使其适合模型的输入。
- **模型选择与训练**：根据问题的特点选择合适的模型架构，并使用训练数据对模型进行训练。
- **模型评估**：使用验证集或测试集对训练好的模型进行评估，衡量模型的性能。
- **调整与优化**：根据评估结果，对模型进行调整，如调整超参数、尝试不同的模型结构等，以提高性能。
- **部署与应用**：将训练好的模型部署到实际应用中，可能需要将模型转换为特定的格式，并与其他系统或模块进行集成。

3.Bert模型的构建？

Bert 模型是一种基于预训练的语言模型，通常使用大量的文本数据进行预训练。构建 Bert 模型的过程包括以下步骤：

- **数据准备**：获取大量的文本数据，例如书籍、文章、网页等。
- **模型架构设计**：选择合适的深度学习架构，如 Transformer 架构，用于处理文本数据。
- **预训练**：使用无监督学习方法，在大量文本上进行预训练，学习语言的统计规律和语义表示。
- **微调**：将预训练好的模型在特定任务上进行微调，例如分类、情感分析等，以适应具体的应用场景。

4.了解哪些模型压缩的方法？详细讲讲一下模型量化

[【深度学习之模型优化】模型剪枝、模型量化、知识蒸馏概述 模型剪枝和量化](#)

模型压缩的方法有很多种，模型量化是其中一种重要的方法。模型量化的原理是将模型的参数从高精度表示转换为低精度表示，例如将 32 位浮点数转换为 8 位整数。这样可以减少模型的存储和计算开销，同时保持一定的准确性。量化的过程通常包括以下步骤：

- **量化策略选择**：确定要使用的量化方法，如整数量化、浮点数量化等。
- **量化参数**：对模型的参数进行量化，将高精度值映射到低精度的量化值。
- **校准**：对量化后的参数进行校准，以补偿量化带来的误差。
- **推理优化**：在量化后的模型上进行推理优化，如使用低精度计算、剪枝等技术。

（1）模型量化

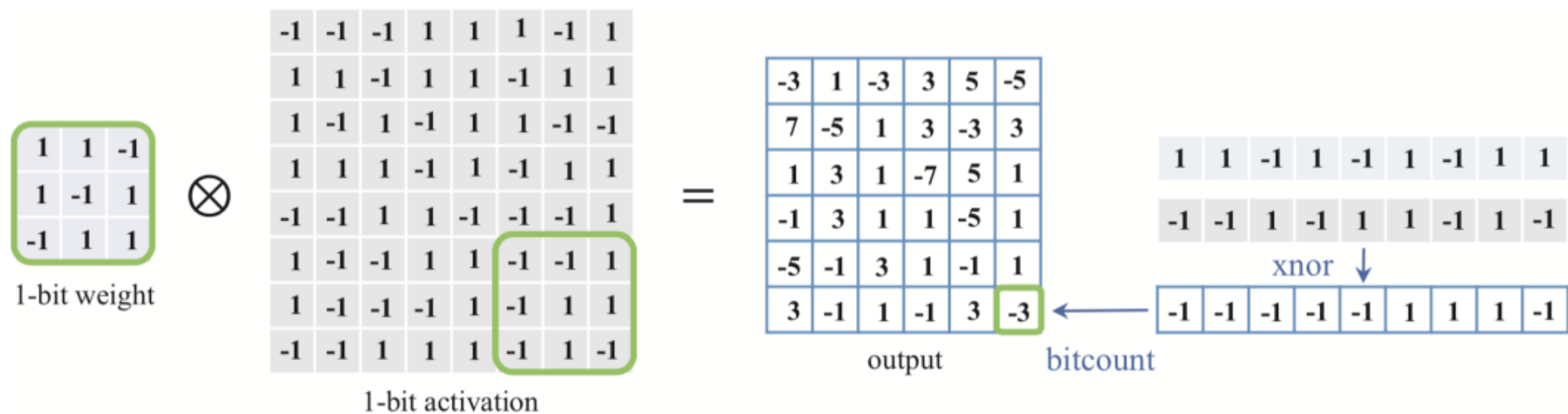
为了保证较高的精度，大部分的科学运算都是采用浮点型进行计算，常见的是32位浮点型和64位浮点型，即float32和double64。

对于深度学习模型来说，乘加计算量是非常大的，往往需要GPU等专用的计算平台才能实现实时运算，这对于端上产品来说是不可接受的，而模型量化是一个有效降低计算量的方法。

量化，即将网络的权值，激活值等从高精度转化成低精度的操作过程，例如将32位浮点数转化成8位整型数int8，同时我们期望转换后的模型准确率与转化前相近。

- **模型量化优势**

- (1) **更小的模型尺寸。**以8bit量化为例，与32bit浮点数相比，我们可以将模型的体积降低为原来的四分之一，这对于模型的存储和更新来说都更有优势。
- (2) **更低的功耗。**移动8bit数据与移动32bit浮点型数据相比，前者比后者高4倍的效率，而在一定程度上内存的使用量与功耗是成正比的。
- (3) **更快的计算速度。**相对于浮点数，大多数处理器都支持8bit数据的更快处理，如果是二值量化，则更有优势。



(2) 模型蒸馏

一般地，大模型往往是单个复杂网络或者是若干网络的集合，拥有良好的性能和泛化能力，而小模型因为网络规模较小，表达能力有限。利用大模型学习到的知识去指导小模型训练，使得小模型具有与大模型相当的性能，但是参数数量大幅降低，从而可以实现模型压缩与加速，就是知识蒸馏与迁移学习在模型优化中的应用。

Hinton等人最早在文章“Distilling the knowledge in a neural network”[1]中提出了知识蒸馏这个概念，其核心思想是一旦复杂网络模型训练完成，便可以用另一种训练方法从复杂模型中提取出来更小的模型，因此知识蒸馏框架通常包含了一个大模型(被称为teacher模型)，和一个小模型(被称为student模型)。

以计算机视觉模型的训练为例，我们经常用在ImageNet上训练的模型作为预训练模型，之所以可以这样做，是因为深度学习模型在网络浅层学习的知识是图像的色彩和边缘等底层信息，某一个数据集学习到的信息也可以应用于其他领域。

知识蒸馏或者说迁移学习的必要性

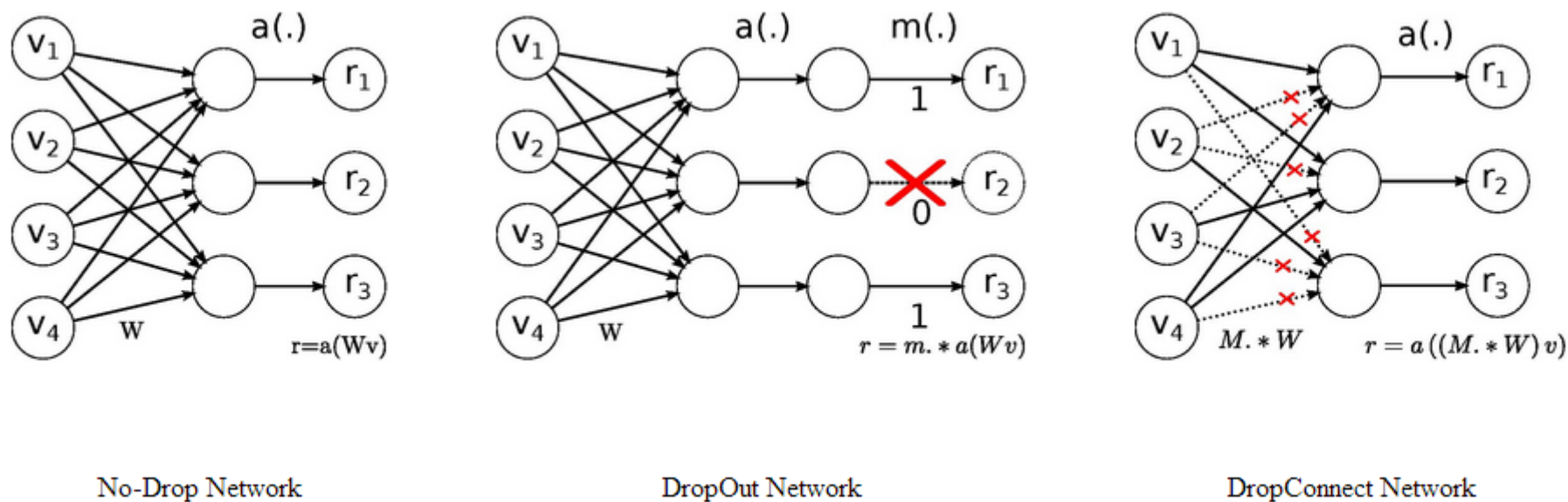
- (1) **数据分布差异。**深度学习模型的训练场景和测试场景往往有分布差异，以自动驾驶领域为例，大部分数据集的采集都是基于白天，光照良好的天气条件下，在这样的数据集上训练的模型，当将其用于黑夜，风雪等场景时，很有可能会无法正常工作，从而使得模型的实用性能非常受限。因此，必须考虑模型从源域到目标域的迁移能力。
- (2) **受限的数据规模。**数据的标注成本是非常高的，导致很多任务只能用少量的标注进行模型的训练。以医学领域为典型代表，数据集的规模并不大，因此在真正专用的模型训练之前往往需要先在通用任务上进行预训练。
- (3) **通用与垂直领域。**虽然我们可以训练许多通用的模型，但是真实需求是非常垂直或者说个性化的，比如ImageNet存在1000类，但是我们可能只需要用到其中若干类。此时就可以基于1000类ImageNet模型进行知识迁移，而不需要完全从头开始训练。

因此，在工业界对知识蒸馏和迁移学习也有着非常强烈的需求。

(3) 模型剪枝

深度学习网络模型从卷积层到全连接层存在着大量冗余的参数，大量神经元激活值趋近于0，将这些神经元去除后可以表现出同样的模型表达能力，这种情况被称为过参数化，而对应的技术则被称为模型剪枝。

Dropout和DropConnect代表着非常经典的模型剪枝技术



- Dropout中随机的将一些神经元的输出置零，这就是神经元剪枝。
- DropConnect则随机的将一些神经元之间的连接置零，使得权重连接矩阵变得稀疏，这便是权重连接剪枝。它们就是最细粒度的剪枝技术，只是这个操作仅仅发生在训练中，对最终的模型不产生影响，因此没有被称为模型剪枝技术。

常用模型剪枝方式：

```
import torch.nn.utils.prune as prune

...
print(list(module.name_parameters()))
print(list(module.name_buffers())) # [weight_mask]
print(module.weight)
print(module.bias)
print(module.state_dict().keys()) # OrderedDict(['weight_orig', 'bias', 'weight_mask'])

prune.remove(module, name='weight')
print(list(module.name_buffers())) # []
print(module.state_dict().keys()) # OrderedDict(['weight', 'bias'])
```

- 对特定网络模块的剪枝 (Pruning Model)

```
# 随机非结构化剪枝
prune.random_unstructured(module, name='weight', amount=0.3)
prune.l1_unstructured(module, name='weight', amount=0.3) # L1剪枝

# 随机结构化剪枝
prune.ln_structured(module, name='weight', amount=0.3, n=2, dim=1) # Ln剪枝
```

- 多参数模块的剪枝 (Pruning multiple parameters)

```
for name, module in model.name_modules():
    if isinstance(module, torch.nn.Conv2d):
        prune.l1_unstructured(module, name='weight', amount=0.2)
    elif isinstance(module, torch.nn.Linear):
        prune.ln_structured(module, name='weight', amount=0.3, n=2, dim=0)
```

- 全局剪枝 (Global pruning)

```
parameters_to_prune = (
    (model.conv1, 'weight'),
    (model.conv2, 'weight'),
    (model.fc1, 'weight'),
    (model.fc2, 'weight'),
    (model.fc3, 'weight'))
prune.global_unstructured(parameters_to_prune, pruning_method=prune.L1Unstructured,
amount=0.2)
# 统计: float(model.fc1.weight == 0)/float(model.fc1.weight.nelement())
```

- 用户自定义剪枝 (Custom pruning)

```
class myself_pruning_method(prune.BasePruningMethod):
    PRUNING_TYPE = 'unstructured'

    def compute_mask(self, t, default_mask):
        mask = default_mask.clone()
        mask.view(-1)[:2] = 0
        return mask

def myself_unstructured_pruning(module, name):
    myself_pruning_method.apply(module, name)
    return module

myself_unstructured_pruning(model.fc3, name='bias')
```

5.了解那些机器学习算法？详细讲一下你用到的随机森林的算法原理

常见的机器学习算法包括决策树、随机森林、支持向量机、朴素贝叶斯、线性回归等。

- 随机森林算法的原理是通过构建多个决策树，并对这些决策树的预测结果进行集成（例如平均或投票）来进行预测。
- 随机森林通过随机选择特征和样本进行训练，从而减少过拟合的风险。

具体来说，随机森林的构建过程包括以下步骤：

- **随机抽样**：从原始数据中随机抽取多个样本，用于构建决策树。
- **构建决策树**：对于每个抽样得到的样本，使用剩余的特征构建决策树。
- **决策树集成**：将多个决策树的预测结果进行集成，得到最终的预测结果。

6.讲一个你认为最能体现你各方面能力的项目？重点讲解一下做项目过程中的优化思路

我可以分享一个项目，在该项目中，我应用了多种技术和方法来优化模型的性能。例如，我采用了以下优化思路：

- **数据增强**：通过对原始数据进行随机旋转、缩放、翻转等操作，增加数据的多样性。
- **超参数调整**：仔细选择合适的超参数，如学习率、正则化参数等，通过网格搜索或随机搜索进行优化。
- **模型集成**：使用多个不同的模型，如多个随机森林或不同的深度学习模型，然后将它们的结果集成起来，提高预测的稳定性和准确性。

- **特征工程**：对数据进行特征提取和选择，选择最相关和有信息量的特征，以提高模型的性能。
- **监控和可视化**：使用监控指标和可视化工具来跟踪模型的性能，及时发现问题并进行调整。

7.你们模型部署是用什么框架的？具体讲讲模型部署过程

模型部署可以使用多种框架，例如 [TensorFlow](#) Lite、PyTorch Mobile、ONNX、NCNN等。模型部署的过程包括将训练好的模型转换为特定框架的格式，进行优化和压缩，然后将模型集成到应用程序中。具体步骤可能包括：

- **模型转换**：将训练好的模型转换为目标框架支持的格式。
- **优化与压缩**：对模型进行优化，如剪枝、量化等，以减少模型的大小和计算量。
- **部署到目标平台**：将模型集成到应用程序中，并在目标平台上进行部署和测试。

8.你们的数据是哪来的？数据内容是什么？

数据的来源会根据具体的项目而有所不同。

- 内部数据集，例如公司内部收集的数据；
- 公开数据集，例如学术研究机构或行业组织发布的数据集；
- 通过爬虫采集数据等。

数据的内容也会因项目的需求而异，例如图像数据、文本数据、音频数据、视频数据等。

9.项目中算法的损失函数是什么？优化器？

项目中使用的损失函数和优化器会根据算法和问题的不同而有所选择。

常见的损失函数包括均方误差、交叉熵等。优化器可以是随机**梯度**下降（SGD）、Adagrad、Adadelta 等。

选择合适的损失函数和优化器可以帮助模型在训练过程中更好地收敛。

- 对于分类问题，交叉熵损失函数常被用于衡量预测结果与真实标签之间的差异；
- 对于回归问题，均方误差可能是一个合适的选择。

优化器的选择则取决于模型的特点和数据的规模。

（1）交叉熵损失函数

交叉熵是信息论中的一个重要概念，主要用于度量两个概率分布间的差异性。在机器学习中，交叉熵表示真实概率分布与预测概率分布之间的差异。交叉熵的值越小，模型预测效果就越好。

以二分类问题为例，假设，正例为，反例为，则交叉熵损失函数表达式为：

$$H(p, q) = - \sum_{i=1}^n p(x_i) \log(q(x_i))$$

$$L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

(2) AdamW优化器

AdamW其实是在Adam的基础上加入了weight decay正则化

在Adam当中，weight decay是直接加入到梯度当中：

$$g_t = g_t + \lambda \theta_{t-1}$$

其中 g_t 是当前step的梯度， θ_{t-1} 是上一个step中的模型权重， λ 是正则化系数。

而在AdamW中，正则化变成了：

$$\theta_t = \theta_{t-1} - \gamma \lambda \theta_{t-1}$$

其中 γ 是学习率。

10.项目中的评估指标有哪些？都是怎么计算的？

[一文读懂准确率、精准率、召回率、ROC、AUC、F1值_准确率公式](#)

评估指标的选择取决于项目的具体目标和问题。

常见的评估指标包括准确率、召回率、F1 值、均方误差、平均绝对误差等。这些指标的计算方式会根据具体的问题和数据而有所不同。

- 在分类问题中，准确率可以用来衡量正确分类的样本比例；
- 在回归问题中，均方误差可以衡量预测值与真实值之间的平均差异。

以下涉及到的参数解释：

- TP (True Positive) ： 真正例
- TN (True Negative) ： 真负例
- FP (False Positive) ： 假正例
- FN (False Negative) ： 假负例

(1) 准确率 Accuracy

准确率的定义是预测正确的结果占总样本的百分比，其公式如下：

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

虽然准确率可以判断总的正确率，但是在样本不平衡 的情况下，并不能作为很好的指标来衡量结果。举个简单的例子，比如在一个总样本中，正样本占90%，负样本占10%，样本是严重不平衡的。对于这种情况，我们只需要将全部样本预测为正样本即可得到90%的高准确率，但实际上我们并没有很用心的分类，只是随便无脑一分而已。这就说明了：由于样本不平衡的问题，导致了得到的高准确率结果含有很大的水分。即如果样本不平衡，准确率就会失效。

(2) 精准率 Precision

精准率 (Precision) 又叫查准率，它是针对预测结果 而言的，它的含义是在所有被预测为正的样本中实际为正的样本的概率，意思就是在预测为正样本的结果中，我们有多少把握可以预测正确，其公式如下：

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

精准率代表对正样本结果中的预测准确程度，而准确率则代表整体的预测准确程度，既包括正样本，也包括负样本。

(3) 召回率 Recall

召回率 (Recall) 又叫查全率，它是针对原样本而言的，它的含义是在实际为正的样本中被预测为正样本的概率，其公式如下：

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

召回率的应用场景：比如拿网贷违约率为例，相对好用户，我们更关心坏用户，不能错放过任何一个坏用户。因为如果我们过多的将坏用户当成好用户，这样后续可能发生的违约金额会远超过好用户偿还的借贷利息金额，造成严重偿失。召回率越高，代表实际坏用户被预测出来的概率越高，它的含义类似：宁可错杀一千，绝不放过一个。

(4) F1值

我们希望精准率和召回率同时都非常高。但实际上这两个指标是一对矛盾体，无法做到双高。如果其中一个非常高，另一个肯定会非常低。选取合适的阈值点要根据实际需求，比如我们想要高的查全率，那么我们会牺牲一些查准率，在保证查全率最高的情况下，查准率也不那么低。在实际情况中，不会有分类器仅仅以精确度(Precision)或者召回率(Recall)作为单一的度量标准，而是使用这两者的调和平均，于是就有了F值(F-Score)，F1分数同时考虑了查准率和查全率，让二者同时达到最高，取一个平衡。

$$\text{F1 score} = 2 \text{ Precision} * \text{Recall} / (\text{Precision} + \text{Recall})$$

11.你们模型服务封装是用什么框架的？具体讲讲模型部署过程

[Flask](#) 是一个轻量级的 [Python](#) Web 框架，常用于 Web 服务的开发。以下是一个简单的 Flask 服务封装流程示例：

1. 部署准备：

- Flask：Python Web 开发中较流行的框架之一，其特点是轻量级。
- Gunicorn：Flask 应用是一个符合 WSGI 规范的 Python 应用，不能单独运行，需要依赖其他的组件提供服务器功能。
- Gevent：Gunicorn 默认使用同步阻塞的网络模型（-k sync），对于高并发的访问并不友好，所以需要使用 gevent 来提高并发量。
- [Docker](#)：可以把它理解为一个“盒子”，有时候会倦于管理项目的部署和维护。如果使用容器封装项目，那么只需要维护一个配置文件即可完成部署需求，包括后续将整个部署的过程完全自动化，部署就会变得更便捷。

2. 构建配置文件：

- 创建 requirements.txt 文件：作为 [python](#) 项目常用到的一个文件，可以让项目中的 Python 环境对依赖包的安装。
- 构建 Gunicorn 配置文件：Flask 应用不能独立运行，需要依赖其他组件提供服务器功能，所以需要构建 Gunicorn 配置文件。
- 创建 Dockerfile 文件：以便构建 Docker 镜像。

3. 封装服务：

- 将 python 服务做成给定参数输入和输出的格式。
- 在 python 里面导入 [flask](#) 包，以及可能用到的 request, jsonify 等。
- app = Flask(name)，app 是 Flask 的实例，它接收包或者模块的名字作为参数，但一般都是传递 `__name__`。
- 在每一个运行的接口服务前面用 `@app.route()` 的方式规定好接口路径和接口请求方式（一般为 post 或者 get）。

- 定义函数具体内容，接收相关参数并进行具体逻辑处理。
- 返回 JSON 格式的内容。
- 用 `app.run()` 来确定主机和端口号。

12.KL散度损失和交叉熵损失有什么区别？

KL散度（Kullback-Leibler Divergence）和交叉熵（Cross Entropy）都是衡量两个概率分布之间差异的指标。它们的区别和联系如下：

- **区别：**
 - **定义不同：** KL散度是用来衡量两个概率分布之间的差异的一种度量方式；交叉熵衡量了模型预测的概率分布与真实概率分布之间的差异。
 - **性质不同：** KL散度不满足交换律和三角不等式；交叉熵满足交换律。
 - **应用不同：** KL散度通常用于比较两个概率分布之间的差异，例如在无监督学习中用于评估生成模型的性能；交叉熵通常用于衡量模型预测和真实标签之间的差异，例如，在分类任务中，交叉熵被用作损失函数，以衡量模型预测的类别分布和真实标签之间的差异。
- **联系：** KL散度可以被用于计算代价，而在特定情况下最小化KL散度等价于最小化交叉熵，而交叉熵的运算更简单，所以用交叉熵来当做代价。

交叉熵通常用于 **监督学习任务** 中，如分类和回归等。在这些任务中，我们有一组输入样本和相应的标签。我们希望训练一个模型，使得模型能够将输入样本映射到正确的标签上。

在这种情况下，我们可以使用交叉熵作为损失函数。假设我们有一个模型预测的输出分布为 p ，真实标签的分布为 q 。那么交叉熵的公式如下：

$$H(p, q) = - \sum_i q_i \log p_i$$

i 表示可能的类别或事件， p_i 和 q_i 分别表示真实概率分布和模型预测的概率分布中类别 i 的概率。

KL散度通常用于 **无监督学习任务** 中，如聚类、降维和生成模型等。在这些任务中，我们没有相应的标签信息，因此无法使用交叉熵来评估模型的性能，所以需要一种方法来衡量模型预测的分布和真实分布之间的差异，这时就可以使用KL散度来衡量模型预测的分布和真实分布之间的差异。KL散度的公式如下：

$$D_{KL}(p||q) = \sum_i p_i \log \frac{p_i}{q_i}$$

i 表示概率分布中的一个可能的事件或状态。 p_i 和 q_i 分别表示真实概率分布和模型预测的概率分布中事件 i 的概率。KL散度衡量了模型预测的概率分布与真实概率分布之间的差异，即模型在预测上的不确定性与真实情况的不确定性之间的差距。

一般情况下：交叉熵通常用于监督学习任务中，KL散度通常用于无监督学习任务中。当我们有相应的标签信息时，应该使用交叉熵来评估模型的性能；当我们没有相应的标签信息时，使用KL散度可以衡量模型预测的分布和真实分布之间的差异。

13.fit(),transform(),fit_transform()有什么区别？

[fit,transform,fit_transform详解](#)

相同点：都是数据预处理中的方法

不同点：

- `fit()`: Method calculates the parameters μ and σ and saves them as internal objects. 解释：简单来说，就是求得训练集X的均值，方差，最大值，最小值,这些训练集X固有的属性。
- `transform()`: Method using these calculated parameters apply the transformation to a particular dataset. 解释：在fit的基础上，进行标准化，降维，归一化等操作（看具体用的是哪个工具，如PCA, StandardScaler等）。
- `fit_transform()`: joins the `fit()` and `transform()` method for transformation of dataset. 解释：fit_transform是fit和transform的组合，既包括了训练又包含了转换。transform()和fit_transform()二者的功能都是对数据进行某种统一处理（比如标准化~N(0,1)，将数据缩放(映射)到某个固定区间，归一化，正则化等）

`fit_transform(trainData)`对部分数据先拟合fit，找到该part的整体指标，如均值、方差、最大值最小值等等（根据具体转换的目的），然后对该全部trainData进行转换transform（补缺失值等），从而实现数据的标准化、归一化等等。